Q17 – DFD Fundamentals (10 marks)

---

a) Define DFD and its purpose (3 marks)

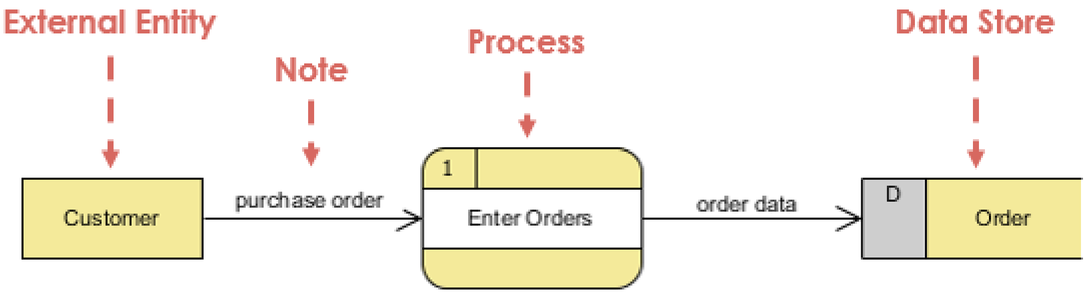
Data Flow Diagram (DFD) is a **graphical representation of how data flows through a system**, showing **inputs, processes, data storage, and outputs**, without describing program logic or implementation details.

Purpose of a DFD

- To understand **what the system does**, not **how it does it**
- To identify **data sources, transformations, and destinations**
- To improve **communication between users and developers**
- To help in **system analysis and requirement specification**

---

b) Explain DFD components (4 marks)



A DFD consists of **four main components**:

1. External Entities (Sources / Sinks)

- Represent **entities outside the system**
- Either **send data to** or **receive data from** the system
- Example: Customer, Bank, Supplier

2. Processes

- Represent **activities that transform input data into output data**
- Must have **at least one input and one output**
- Example: *Process Order*, *Calculate Salary*

3. Data Stores

- Represent **places where data is stored**
- Can be files, databases, or tables
- Example: *Customer Records*, *Orders File*

4. Data Flows

- Represent **movement of data** between components
- Shown using **arrows**
- Must be **clearly named**
- Example: *Order Details*, *Payment Info*

---

c) DFD rules with violations (3 marks)

Rule 1: A process must have both input and output

❌ **Violation:** Process with only input or only output

Rule 2: Data cannot flow directly between two data stores

❌ **Violation:** Data store → Data store (must go through a process)

Rule 3: Data cannot flow directly from an external entity to a data store

❌ **Violation:** External Entity → Data Store without a process

---

Rule 4: Data cannot flow directly between two external entities

❌ **Violation:** External Entity → External Entity

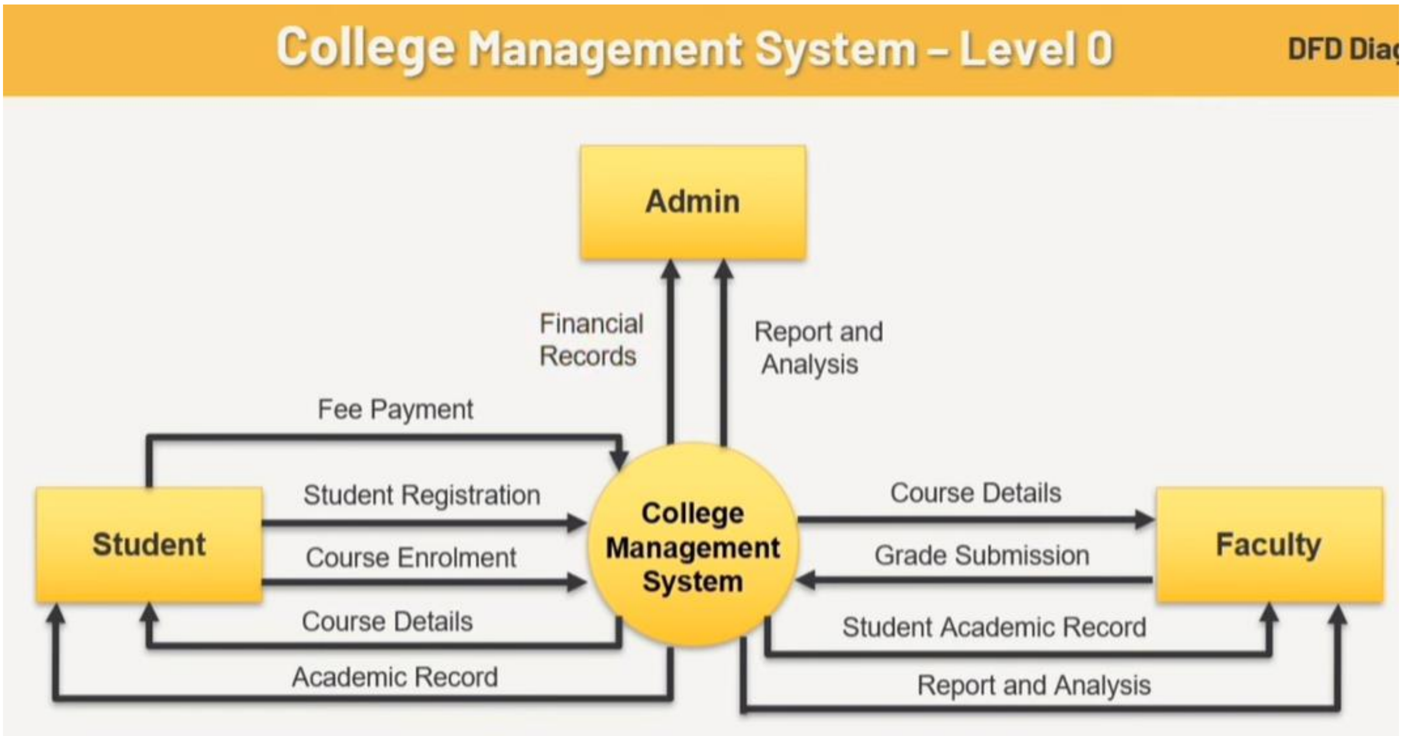
Rule 5: Every data flow must be meaningful and named

❌ **Violation:** Arrow without a label or unclear data meaning

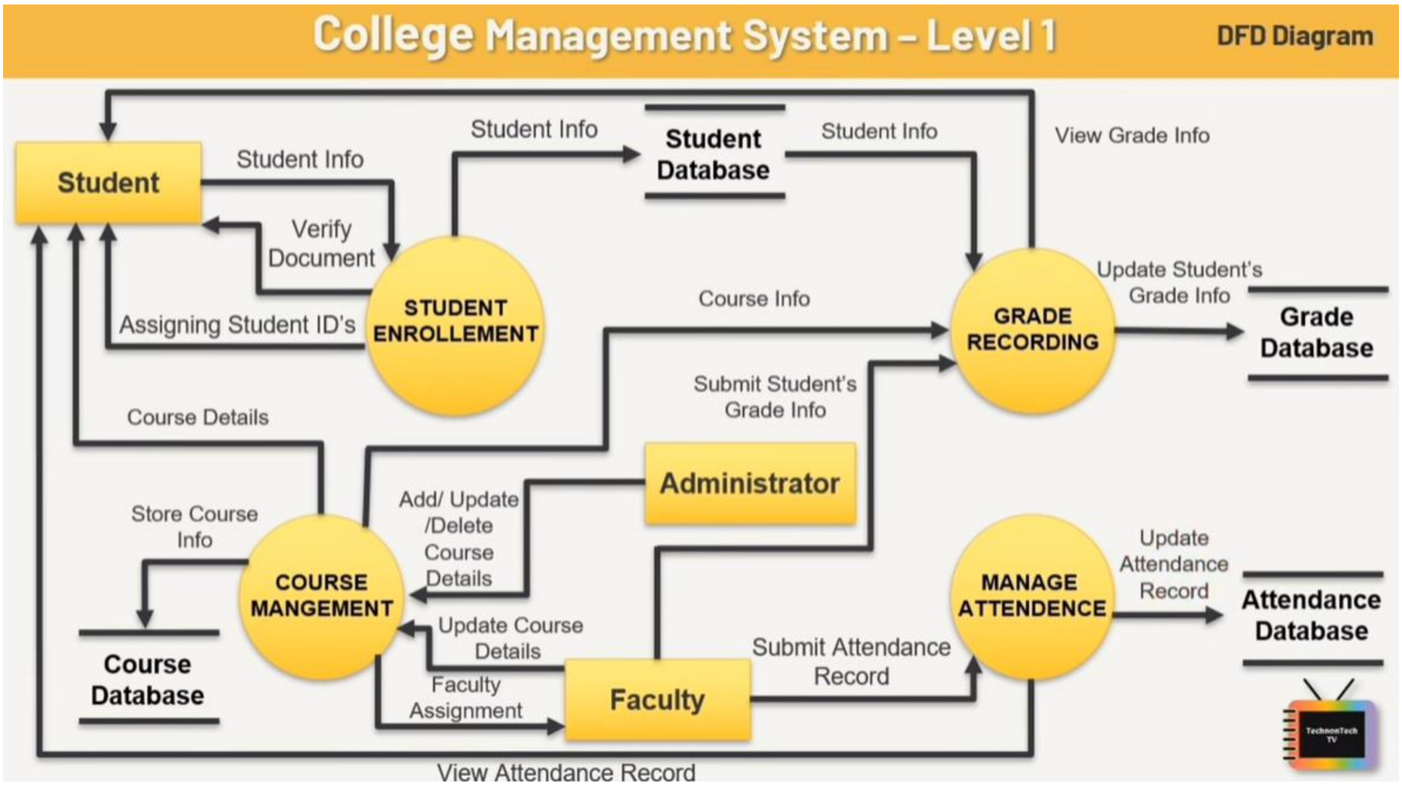
---

DFD example

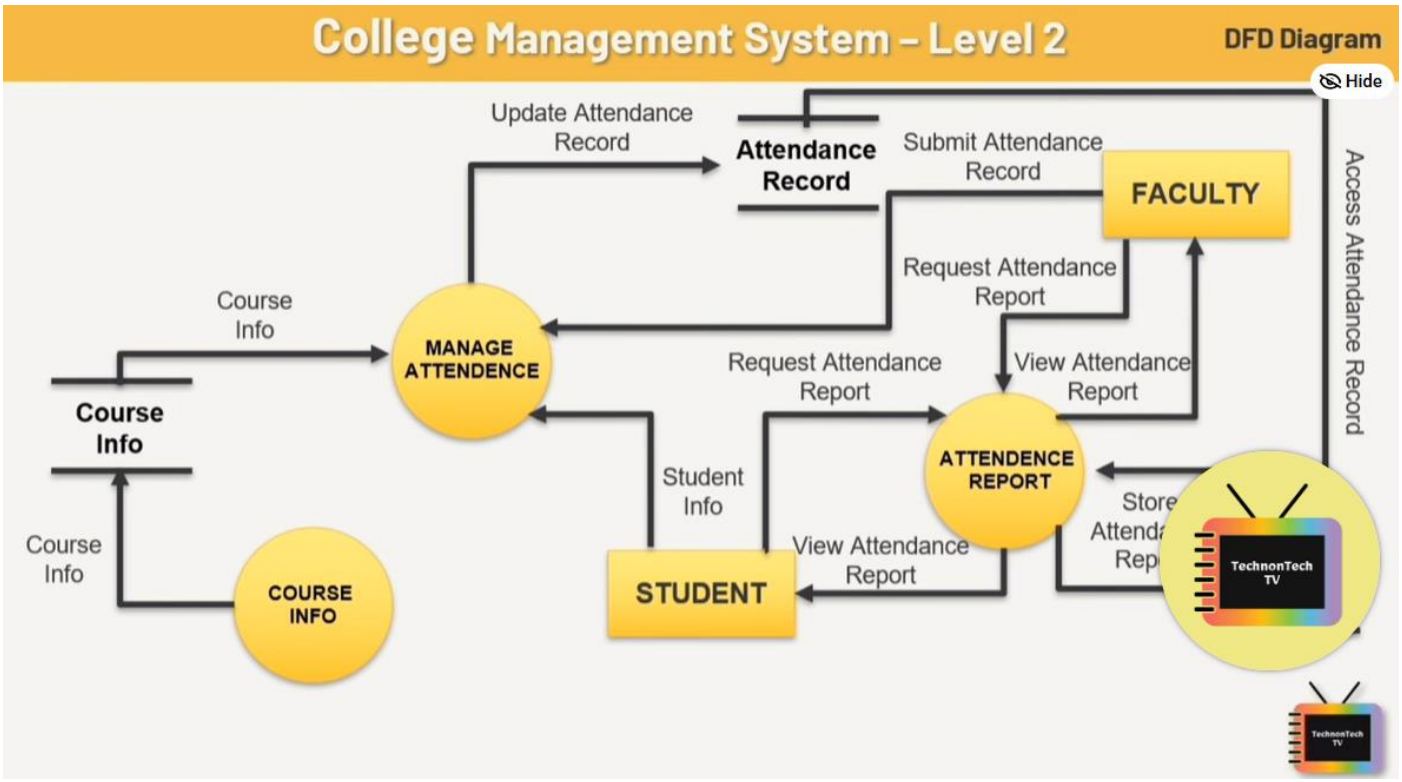
0 level DFD – Context Diagram



1 level DFD



2 level DFD



Aspect	Level 0 DFD	Level 1 DFD	Level 2 DFD
Also called	Context Diagram	Top-level DFD	Detailed DFD
Number of processes	1	Many	Many (more detailed)
Shows system boundary	✅ Yes	❌ No	❌ No
Shows data stores	❌ No	✅ Yes	✅ Yes
Level of detail	Very low	Medium	High
Purpose	System overview	Functional breakdown	Detailed process logic
Decomposition of	—	Level 0	Level 1

1️⃣ Levelling

Definition

Levelling is the process of breaking down a DFD into multiple levels to show increasing detail, starting from a high-level view and moving to more detailed views.

Explanation

- **The system is first shown as a single process (Level 0).**

- That process is decomposed into smaller sub-processes (Level 1).
- Each sub-process can be further decomposed (Level 2, Level 3, etc.).

Purpose

- To manage system complexity
- To improve clarity and understanding
- To show the system step by step, from general to detailed

Example

Level 0 → *Order Processing System*
Level 1 → *Validate Order*, *Process Payment*, *Update Inventory*
Level 2 → *Check Stock*, *Reserve Items*, *Generate Invoice*

2️⃣ Balancing

Definition

Balancing means that data inputs and outputs must remain consistent when a process is decomposed from one DFD level to the next.

Explanation

- When a process in a higher-level DFD is expanded in a lower-level DFD:
  - No new inputs or outputs can be added
  - No existing inputs or outputs can be removed
- The external data flows must match exactly

Purpose

- To maintain logical consistency
- To ensure correct system representation
- To avoid missing or extra data flows

Example

If Level 0 shows:

- Input: *Order Details*
- Output: *Invoice*

Then Level 1 must also show:

- Input: *Order Details*
- Output: *Invoice*

❌ Adding *Payment Confirmation* only in Level 1 → Balancing violation